

머신러닝 4 교시

— 교차 검증과 GridSearchCV —

(Cross-Validation · Stratified K-Fold · GridSearch — iris 데이터로 배우는 모델 평가의 정석)

학습 목표

- 학습 데이터만으로 평가하면 정확도가 1.0 이 나오는 이유, 그리고 그것이 "좋은 모델"의 신호가 결코 아니라는 점을 설명할 수 있다.
- 단일 train_test_split 의 한계와 교차검증이 필요한 이유를 설명할 수 있다.
- train_test_split() 으로 홀드아웃(hold-out, 데이터셋 분리)를 수행하고 데이터셋분리·학습·예측·평가의 표준 흐름대로 코딩할 수 있다.
- KFold 와 StratifiedKFold 의 차이를 "클래스 분포" 관점에서 설명하고, 분류 문제에서 왜 후자를 기본으로 써야 하는지 말할 수 있다.
- cross_val_score() 한 줄로 K-Fold CV 전체를 수행하고 결과의 평균·표준편차를 해석할 수 있다.
- GridSearchCV 로 하이퍼파라미터를 자동 튜닝할 수 있다.
- GridSearchCV 로 best_params_, best_score_, best_estimator_ 를 활용해 최종 모델을 만들 수 있다.

0. 데이터셋 소개 — iris

이번 단원에서 사용하는 데이터는 R. A. Fisher 가 1936 년에 정리한 "iris" 데이터셋이다. 사이킷런이 기본 내장하고 있어서 한 줄로 불러올 수 있고, 클래스가 3 개·샘플이 150 개로 작아서 "평가 절차 자체"를 배우기에 가장 좋다.

- 샘플 수: 150 (각 클래스 50 개씩 정확히 균형)
 - 특성: sepal length / sepal width / petal length / petal width 의 4 개 수치형
 - 클래스(타깃): setosa(0), versicolor(1), virginica(2) — 정수 라벨로 인코딩되어 있다.
 - 주의: iris.data 는 "클래스 순서대로 정렬"되어 있다. 그래서 KFold(shuffle=False) 로 그냥 자르면 한 폴드에 한 클래스만 몰리는 사고가 생긴다 — 뒤에서 보게 될 것이다.
- 세 종(species) 의 실제 모습은 다음과 같다.



Iris setosa



Iris versicolor



Iris virginica

1. "학습 데이터로 평가하면 안 된다" — 첫 번째 함정

머신러닝을 처음 배울 때 가장 자주 저지르는 실수는 다음과 같다.

- "전체 데이터로 모델을 학습시킨 뒤, 같은 데이터로 정확도를 잴다."

코드 자체는 자연스러워 보인다.

```

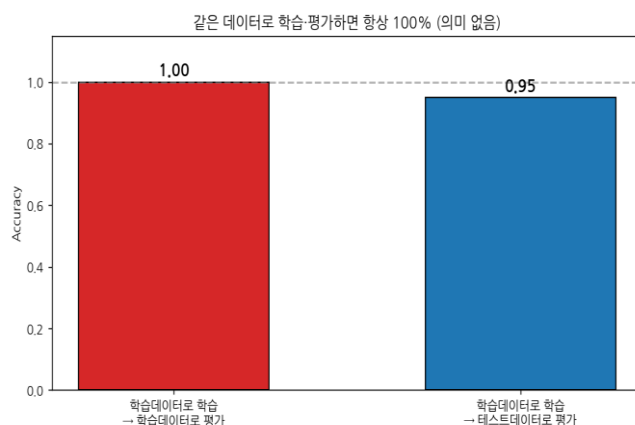
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()
dt_clf = DecisionTreeClassifier()
X = train_data = iris.data
y = train_label = iris.target

# 같은 데이터로 학습·예측
dt_clf.fit(train_data, train_label)
pred = dt_clf.predict(train_data)
print('예측 정확도:', accuracy_score(train_label, pred))
# → 1.0

```

결과는 1.0 — 완벽한 정확도가 나온다. 그러나 이것은 "모델이 똑똑하다"가 아니라 "모델이 답을 외웠다"는 뜻이다. 결정트리는 분기를 계속 늘려가면서 학습 샘플 하나하나에 "이 샘플은 이 답이야"라고 메모해 둘 수 있다. 새로 들어온 데이터에 대해서는 그 메모가 통하지 않으므로, 학습 데이터에서의 1.0은 미래 성능에 대해 어떤 정보도 주지 못한다.



[그림 3] 같은 데이터로 학습·평가하면 정확도가 1.0이 되지만, 한 번도 안 본 테스트 데이터로 평가하면 점수가 "진짜 일반화 성능"으로 떨어진다. 우리가 알고 싶은 것은 오른쪽의 숫자이다.

이 "학습데이터=평가데이터"의 결과는 모델 평가에서 절대 사용하면 안 되는 숫자이다. 이를 피하기 위한 첫 번째 해결책이 다음 절의 `train_test_split()` 이다.

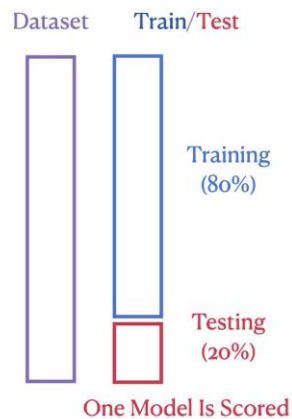
2. 일반적인 학습·평가 절차 — train_test_split

가장 단순한 해결책은 데이터를 두 덩어리로 자르는 것이다.

- 학습용 (train) — 모델에게 "이걸 보고 배워라" 라고 주는 데이터
- 테스트용 (test) — 학습이 끝난 모델에게 "이건 한 번도 못 본 거야, 맞춰 봐" 라고 평가하는 데이터

이 방식(= train set 과 test set 으로 나누는 방식)을 홀드아웃(hold-out) 검증이라고 부른다.

사이킷런은 train_test_split() 으로 이를 한 줄에 해 준다.



[그림 4]

train_test_split(test_size=0.2) 의 개념. 전체 데이터 중 80%로 모델을 학습시키고, 손대지 않은 나머지 20%로 단 한 번 평가한다.

2.1 코드

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris_data = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris_data.data, iris_data.target,
    test_size=0.3, random_state=121, stratify=y)

dt_clf = DecisionTreeClassifier()
dt_clf.fit(X_train, y_train)
pred = dt_clf.predict(X_test)
print('예측 정확도: {0:.4f}'.format(accuracy_score(y_test, pred)))
# 예: 0.9556
```

핵심 인자

- test_size=0.3 — 테스트셋 비율. 일반적으로 0.2~0.3 을 쓴다. 데이터가 매우 작으면 더 큰 테스트 비율을, 데이터가 매우 크면 0.1 이하도 쓸 만하다.
- random_state=121 — 분할에 쓰이는 난수 시드. 같은 값을 주면 누가 실행해도 같은 분할이 나와 결과가 재현된다.
- stratify=y (선택) — 학습 셋과 테스트 셋 내의 **클래스 비율을 원본 데이터와 동일하게 유지**해 줌. 분류 문제에서 클래스 비율을 학습/테스트에 동일하게 유지하고 싶을 때 지정. 사실상 필수. 기본값이 None 이라 y 로 지정.

2.2 pandas 로도 해 보기

`train_test_split()` 은 NumPy ndarray 뿐 아니라 pandas DataFrame / Series 도 그대로 처리한다. 컬럼 정보가 남기 때문에 분석·전처리에는 일반적으로 DataFrame 형태가 유리하다.

```
import pandas as pd

iris_df = pd.DataFrame(iris_data.data, columns=iris_data.feature_names)
iris_df['target'] = iris_data.target

ftr_df = iris_df.iloc[:, :-1]      # 마지막 열 제외한 특성
tgt_df = iris_df.iloc[:, -1]      # 마지막 열 = target

X_train, X_test, y_train, y_test = train_test_split(
    ftr_df, tgt_df, test_size=0.3, random_state=121)

print(type(X_train), type(y_train))
# <class 'pandas.core.frame.DataFrame'> <class 'pandas.core.series.Series'>

dt_clf = DecisionTreeClassifier()
dt_clf.fit(X_train, y_train)
pred = dt_clf.predict(X_test)
print('예측 정확도: {0:.4f}'.format(accuracy_score(y_test, pred)))
```

그래도 홀드아웃 한 방식엔 약점이 남는다 — "분할 한 번" 으로 얻은 점수이기 때문에, 운 좋게/ 나쁘게 잘린 한 가지 결과에 좌우된다. 이 문제를 정면으로 푸는 도구가 교차 검증이다.

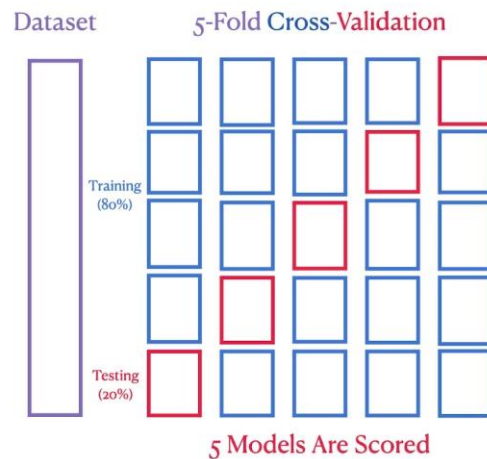
3. 교차 검증 (Cross-Validation) — 왜, 그리고 어떻게

한 번의 train/test 분할만으로는 다음과 같은 아쉬움이 남는다.

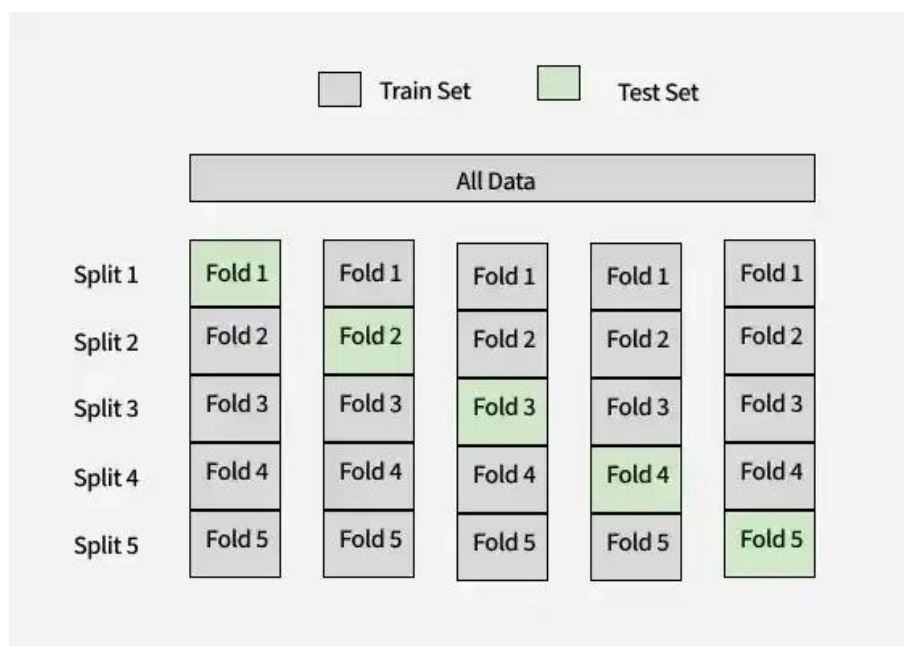
- 귀한(=부족한) 데이터를 한번만 쓰고 말기에는 아깝다. .
- 한 번만 모델을 돌려서 나온 성능을 믿을 수 없다.
- "단 한 번" 잘린 결과라서 분할 운에 점수가 크게 바뀐다 (어떤 random_state 로 나누었는지에 따라 결과가 1~3%p 달라질 수 있다.).
- 테스트셋으로 빼둔 데이터는 학습에 쓰지 못한다 — 데이터가 적을수록 손해가 크다.

그래서 생각한 아이디어는 단순하다. 한 데이터를 여러 번 쓰는 것이다.

데이터를 K 개의 덩어리(폴드, fold)로 나눈 뒤, K 번 학습/평가를 반복하면서 매번 다른 폴드를 "검증세트" 로 사용한다. 점수 K 개의 평균이 모델의 일반화 성능 추정치이다. 이런 방식을 교차검정이라고 한다.



[그림 5] 5-Fold Cross-Validation 의 개념도. 데이터를 5 등분한 뒤, 각 반복마다 1 개 폴드만 검증세트(빨강)로, 나머지 4 개를 학습세트로 사용한다. 결과적으로 5 개의 점수가 나오고 그 평균을 본다



[그림 6] K-Fold 의 또 다른 시각화. "한 번씩 모든 폴드가 검증세트가 된다" 는 점이 핵심이다
— 모든 데이터를 "학습에도, 평가에도" 쓰는 셈이다.

이 그림에서 주의할 것은 test set 을 제외한 train set 에 대해서만 cross validation 을 한다는 점이다. 그림에서 나오는 Dataset 은 전체 데이터셋을 뜻하는 게 아니라, 80% 에 해당하는 trainset 를 지칭한다.

4. KFold — 사이킷런으로 직접 돌려 보기

사이킷런의 KFold 객체는 "각 반복에서 사용할 학습/검증 인덱스" 를 만들어 준다. `.split(features)` 가 K 번의 (train_index, test_index) 튜플을 차례로 내놓는 제너레이터(generator) 이다.

5-Fold Cross-Validation 모식도

Iter 1	검증	학습	학습	학습	학습
Iter 2	학습	검증	학습	학습	학습
Iter 3	학습	학습	검증	학습	학습
Iter 4	학습	학습	학습	검증	학습
Iter 5	학습	학습	학습	학습	검증

[그림 7] KFold(n_splits=5) 의 진행. Iter 1 에서는 첫 폴드가 검증, 나머지가 학습. Iter 2 에서는 두 번째 폴드가 검증, ... 이렇게 K 번 학습/평가를 반복한다.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn.model_selection import KFold
from sklearn.datasets import load_iris
import numpy as np

iris = load_iris()
features = iris.data
label = iris.target
dt_clf = DecisionTreeClassifier(random_state=156)

kfold = KFold(n_splits=5)
cv_accuracy = []

for train_index, test_index in kfold.split(features):
    X_train, X_test = features[train_index], features[test_index]
    y_train, y_test = label[train_index], label[test_index]

    dt_clf.fit(X_train, y_train)
    pred = dt_clf.predict(X_test)

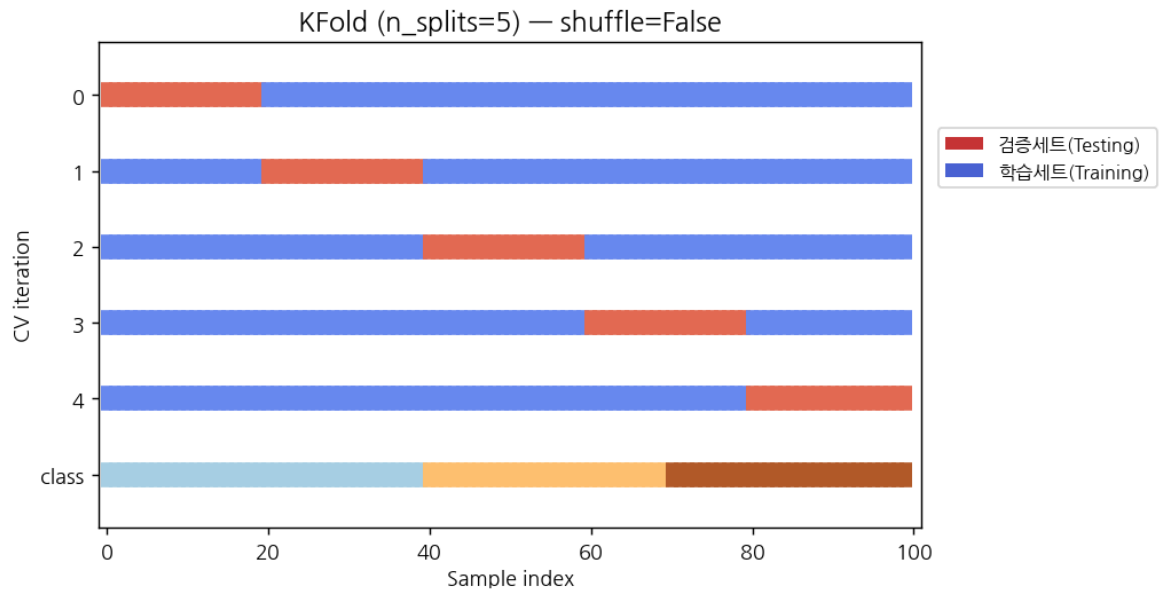
    accuracy = accuracy_score(y_test, pred)
    cv_accuracy.append(accuracy)

print('개별 검증 정확도:', cv_accuracy)
print('평균 검증 정확도:', np.mean(cv_accuracy))
```

실행 결과(예시)는 다음과 같다. 5 개 폴드 각각의 정확도와 그 평균이 출력된다.

```
개별 검증 정확도: [1.0, 0.9667, 0.8667, 0.9333, 0.7333]
평균 검증 정확도: 0.9
```

개별 점수가 0.73 ~ 1.0 으로 들쭉날쭉한 점에 주목해야 한다. 평균 0.9 라는 숫자는 "이 모델이 항상 90%" 라는 뜻이 아니라, "이 데이터에 대한 평균적인 일반화 성능 추정치" 이다. 그리고 마지막 폴드의 점수가 유독 낮은 것은 우연이 아니라 "클래스 분포 편향" 이 원인이다 — 다음 절에서 직접 확인한다.



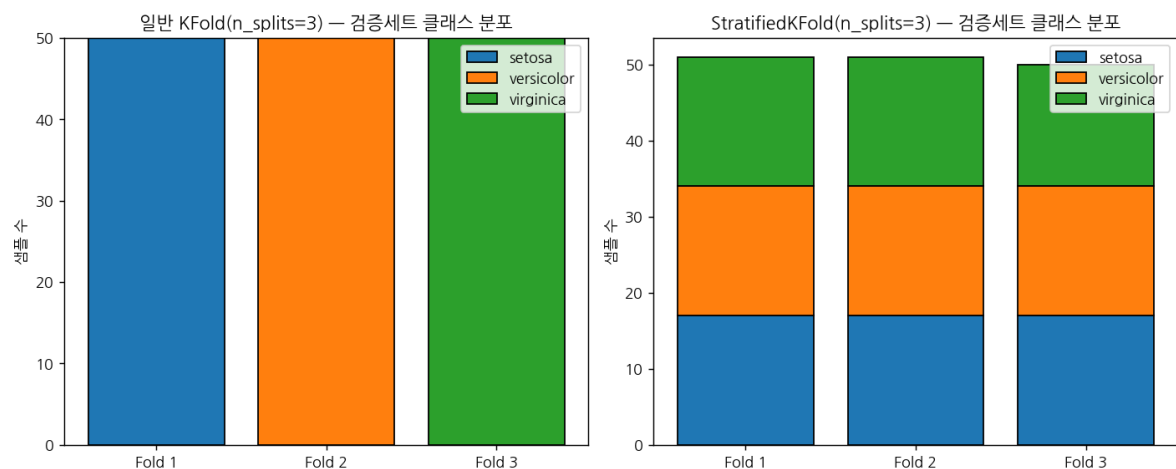
[그림 8] KFold 시각화. 각 클래스가 연속적으로 있는 데이터에 일반 KFold 를 적용하면 "검증세트가 한 클래스 위주" 가 되기 쉽다는 것이 한눈에 보인다.

5. Stratified K-Fold — 분류 문제의 기본값

iris 데이터셋은 setosa 50 개 → versicolor 50 개 → virginica 50 개 순으로 정렬되어 있다. 여기에 그냥 KFold(n_splits=3) 을 쓰면 어떻게 되는가?

일반 KFold 의 폴드별 클래스 분포 (개념)
 Fold 1 검증세트 → setosa 50 개
 Fold 2 검증세트 → versicolor 50 개
 Fold 3 검증세트 → virginica 50 개

이런 상태에서 학습/평가를 하면 "학습세트에 없는 클래스를 검증세트에서 맞춰야 하는" 상황이 벌어진다 — 정확도가 0 이 나와도 이상하지 않다. 실제로 위 KFold 결과의 마지막 폴드 점수가 0.73 으로 떨어진 이유가 바로 이것이다.



[그림 9] 일반 KFold vs StratifiedKFold 의 검증세트 클래스 분포 비교. 왼쪽(KFold) 은 한 폴드에 한 클래스만 들어가지만, 오른쪽(StratifiedKFold) 은 모든 폴드가 세 클래스를 거의 같은 비율로 가진다.

StratifiedKFold 는 이름 그대로 "계층(stratum) 별로 균등하게" 폴드를 만든다. 분류 문제에서는 거의 항상 이쪽을 쓴다고 보면 된다. 호출 방식이 KFold 와 거의 똑같은데, .split() 호출 시 X 뿐 아니라 y(레이블) 도 함께 넘겨주어야 한다는 점만 다르다.

```
from sklearn.model_selection import StratifiedKFold
import numpy as np

skfold      = StratifiedKFold(n_splits=3)
cv_accuracy = []

# .split( ) 에 features 와 label 을 함께 넣는다는 점이 핵심
for train_index, test_index in skfold.split(features, label):
    X_train, X_test = features[train_index], features[test_index]
    y_train, y_test = label[train_index], label[test_index]

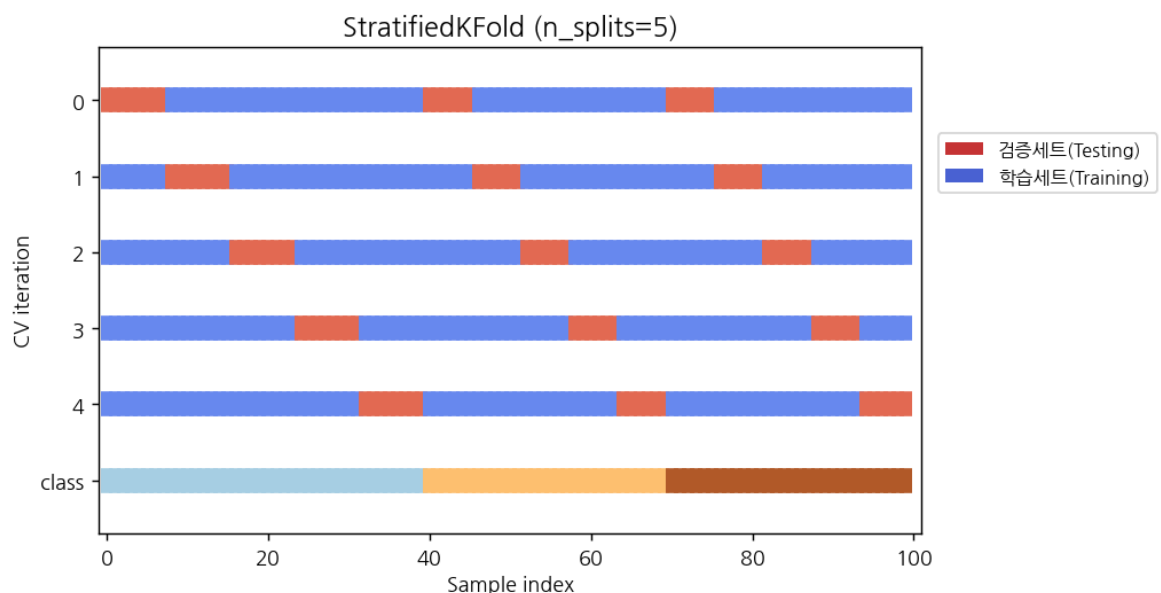
    dt_clf.fit(X_train, y_train)
    pred = dt_clf.predict(X_test)
    cv_accuracy.append(accuracy_score(y_test, pred))

print('교차 검증별 정확도:', np.round(cv_accuracy, 4))
print('평균 검증 정확도:', np.mean(cv_accuracy))
```

실행 결과(예시).

```
교차 검증별 정확도: [0.98 0.94 0.98]
평균 검증 정확도: 0.9666666666666667
```

개별 점수가 0.94 ~ 0.98 사이로 매우 안정적이다 — 즉 "이 모델은 어떤 분할에서도 비슷한 성능을 낸다" 는 신뢰할 만한 신호가 된다.



[그림 10] StratifiedKFold 의 시각화. 각 반복(빨강=검증세트)이 모든 클래스 색깔을 골고루 가져가는 것이 보인다. KFold 와 비교해 보면 차이가 명확하다.

6. cross_val_score — 한 줄로 끝내기

위 "for 문으로 KFold/StratifiedKFold 돌리기" 는 사이킷런이 이미 cross_val_score() 라는 함수 한 줄로 묶어 두었다. 사용법이 너무 깔끔해서 실무에서는 보통 이쪽을 쓴다.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score
from sklearn.datasets import load_iris
import numpy as np

iris_data = load_iris()
dt_clf = DecisionTreeClassifier(random_state=156)

data = iris_data.data
label = iris_data.target

# 평가지표=정확도, 폴드=3 개
scores = cross_val_score(dt_clf, data, label,
                          scoring='accuracy', cv=3)

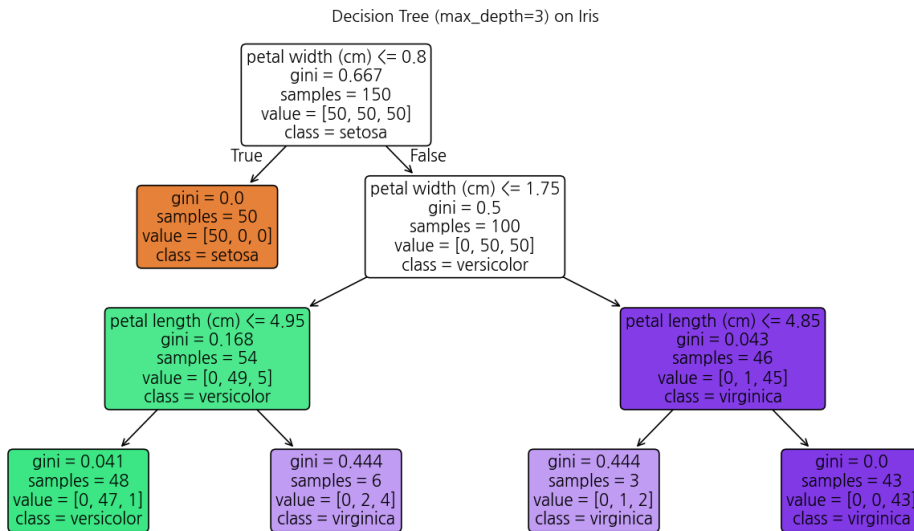
print('교차 검증별 정확도:', np.round(scores, 4))
print('평균 검증 정확도:', np.round(np.mean(scores), 4))
```

이 함수는 분류기인지(자동으로 StratifiedKFold), 회귀기인지(KFold)를 보고 적절한 폴드 객체를 자동으로 골라 준다. 결과는 위 5 절의 Stratified 코드와 동일하게 나온다.

- scoring= 인자에 'accuracy', 'f1_macro', 'roc_auc', 'neg_mean_squared_error' 같은 문자열을 넣어 평가지표를 바꿀 수 있다.
- cv= 자리에 정수가 아닌 "KFold(n_splits=5, shuffle=True, random_state=42)" 같은 객체를 직접 넣어 세밀히 제어할 수도 있다.
- 반환값은 numpy 배열이므로 .mean(), .std() 로 평균과 변동성을 함께 보는 습관을 들이면 좋다.

7. GridSearchCV — 하이퍼파라미터 탐색

지금까지의 코드는 모두 DecisionTreeClassifier() 를 "기본 설정" 으로 사용했다. 그러나 결정트리에는 우리가 직접 정해야 하는 설정값들이 있다 — 이것을 하이퍼파라미터(hyperparameter) 라고 부른다.



[그림 11] iris 데이터에 학습된 결정트리(max_depth=3)의 모습. 분기 조건(petal width ≤ 0.8 등)과 gini 불순도, 샘플 수, 추정 클래스가 노드마다 표시된다. max_depth와 min_samples_split을 바꾸면 이 트리의 모양 자체가 달라진다.

- max_depth — 트리를 얼마나 깊게 키울지. 작으면 단순(과소적합), 크면 복잡(과대적합).
- min_samples_split — 노드를 분할하기 위해 필요한 최소 샘플 수. 크면 트리가 덜 자란다.
- min_samples_leaf — 잎(leaf) 노드가 가져야 할 최소 샘플 수. 노이즈에 과민하지 않게 만든다.

이런 하이퍼파라미터의 "좋은 값"은 데이터마다 다르므로, 후보 값들을 직접 시도해 보면서 가장 좋은 조합을 찾아야 한다.

우리의 목적은 성능을 최대로 만드는 각 하이퍼파라미터들의 값을 찾아내는 것이다. 이 조합을 찾으려면 해당하는 경우의 수를 모두 시도해 보는 수 밖에는 없는 것으로 보인다. 그래서 실제로 해 보기로 한다.

모든 경우의 수를 나타내는 네모판을 만들고 탐색을 할 것이다. 이를 격자 탐색(Grid Search)이라 부른다. 또 한가지 중요한 것은 파라미터들의 조합 한가지에 대하여 한번만 실행하는 것이 아니라 교차검증(Cross Validation)을 한다는 점이다. 그래서 GridSearchCV라고 한다.

사이킷런은 GridSearchCV라는 클래스로 한 번에 해 준다 — 이름에 CV가 붙은 이유는 "각 조합을 교차 검증으로 평가하기 때문"이다.

7.1 GridSearchCV 호출

```

from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.metrics import accuracy_score
import pandas as pd

iris = load_iris()
  
```

```

X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=121)

dtree = DecisionTreeClassifier()

# 후보 하이퍼파라미터들을 사전 형태로 정의
parameters = {
    'max_depth': [1, 2, 3],
    'min_samples_split': [2, 3],
}

# cv=3 → 각 조합을 3-fold CV 로 평가
# refit=True (기본값) → 최적 조합으로 전체 train 데이터에 재학습
grid_dtree = GridSearchCV(
    dtree,
    param_grid=parameters,
    cv=3,
    refit=True,
    return_train_score=True,
)

grid_dtree.fit(X_train, y_train)

```

위 호출에서 GridSearchCV 는 다음을 자동으로 처리한다.

- $\text{max_depth} \times \text{min_samples_split} = 3 \times 2 = 6$ 개의 조합을 만든다.
- 각 조합에 대해 학습데이터를 3-fold 로 나눠 학습/평가를 수행한다 — 즉 $6 \times 3 = 18$ 번의 학습이 일어난다.
- 18 번의 결과를 정리해 `cv_results_` 라는 사전에 저장한다.
- 가장 좋은 조합으로 `X_train` 전체에 다시 한 번 학습 시켜 `best_estimator_` 에 보관한다 (`refit=True`).
- `grid_dtree` 에는 찾아낸 최고의 조합과 모델이 저장되어 있다.

7.2 결과 살펴보기 — `cv_results_`

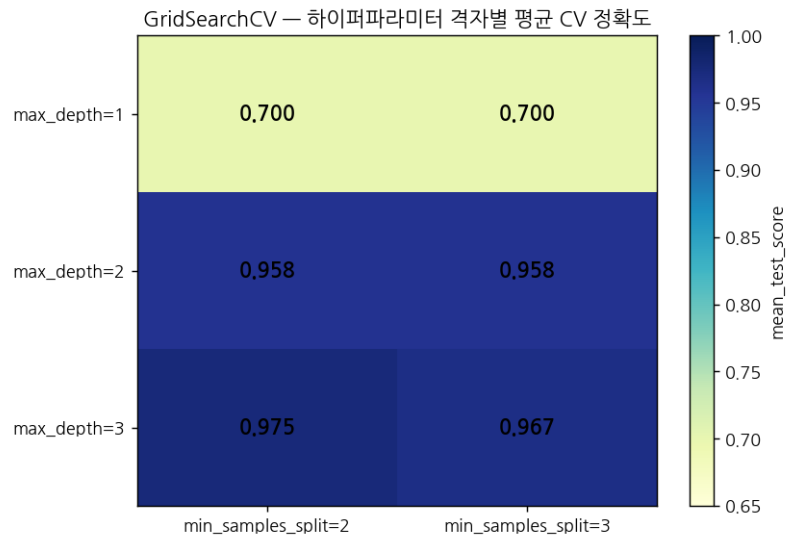
```

# cv_results_ 는 사전. DataFrame 으로 만들어 보면 한눈에 보인다.
scores_df = pd.DataFrame(grid_dtree.cv_results_)

# 주요 컬럼만 골라 보기
scores_df[['params',
           'mean_test_score',
           'rank_test_score',
           'split0_test_score',
           'split1_test_score',
           'split2_test_score']]

```

이렇게 만들어진 표를 "하이퍼파라미터 × 평균 점수" 의 히트맵으로 시각화하면 어떤 조합이 더 좋은지가 직관적으로 보인다.



[그림 12] 위 GridSearchCV 결과의 평균 검증 정확도를 (max_depth, min_samples_split) 격자로 시각화한 예. max_depth=1은 분할이 부족해 0.70에 머무르고, max_depth=3이 되면 0.97대로 올라간다.

7.3 최적 조합 — best_params_ / best_score_ / best_estimator_

grid_dtree에는 찾아낸 최고의 조합이 저장되어 있으므로 출력만 하면 된다.

```
print('GridSearchCV 최적 파라미터:', grid_dtree.best_params_)
print('GridSearchCV 최고 정확도: {0:.4f}'.format(grid_dtree.best_score_))

# 출력 예시
# GridSearchCV 최적 파라미터: {'max_depth': 3, 'min_samples_split': 2}
# GridSearchCV 최고 정확도: 0.9750
```

주의: best_score_는 "학습데이터의 3-fold CV 평균" 이므로 우리가 마지막에 보고할 "테스트 데이터 정확도"와 다르다. 다음 단계에서 따로 측정한다.

7.4 테스트 데이터로 최종 평가

grid_dtree에는 찾아낸 최고의 '모델'도 저장되어 있으므로 예측만 하면 된다.

```
# refit=True 였으므로 GridSearchCV 객체 자체가
# 최적 모델로 이미 재학습된 상태 → 그대로 predict 가능
pred = grid_dtree.predict(X_test)
print('테스트 데이터 세트 정확도: {0:.4f}'.format(
    accuracy_score(y_test, pred)))

# 같은 결과 - best_estimator_ 로 명시적으로 예측해도 동일
pred2 = grid_dtree.best_estimator_.predict(X_test)
print('테스트 데이터 세트 정확도: {0:.4f}'.format(
    accuracy_score(y_test, pred2)))

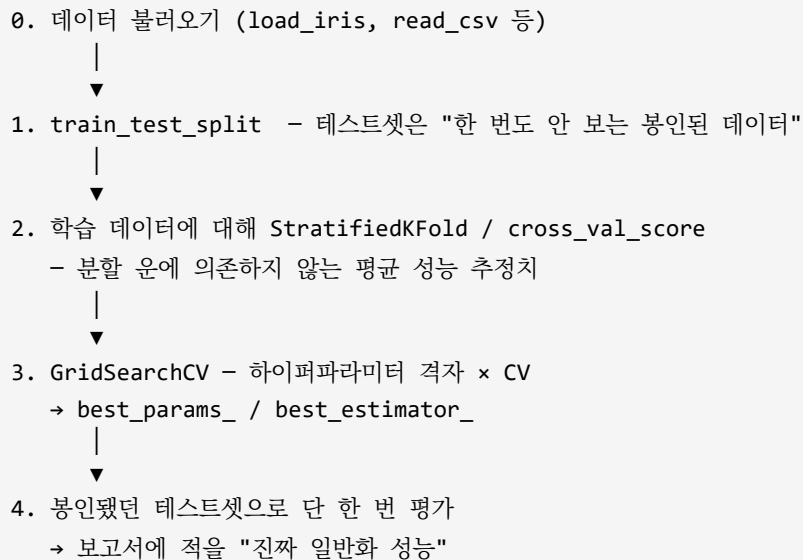
# 둘 다 같은 숫자, 예: 0.9667
```

이 마지막 "테스트 데이터 세트 정확도"가 우리가 보고서에 "이 모델의 일반화 성능"으로 적을 수 있는 단 하나의 숫자이다. 학습 데이터로 얻은 best_score_(0.9750)와 비슷한 값이 나오면

"CV 가 일반화 성능을 잘 추정했다" 는 좋은 신호이고, 둘이 크게 벌어지면 분할 운/과적합/데이터 누수 등을 의심해야 한다.

8. 단원 정리 — 한 장 요약

모델 평가 절차 "표준 흐름"



- "학습 데이터로 평가" 의 결과 1.0 는 기뻐할 일이 아니다. 성능이 좋아 보일수록 의심하라.
- 분류 문제에서는 KFold 보다 StratifiedKFold 를 기본으로 쓴다. cross_val_score 는 분류기/회귀기를 보고 알아서 골라 준다.
- GridSearchCV 의 best_score_ 는 "학습데이터의 CV 평균" 이고, "테스트셋 정확도" 는 .predict() 로 따로 측정한다. 둘은 다르다 — 비교해 보는 것이 중요하다.
- 탐색할 조합 수가 너무 많으면 GridSearchCV 대신 RandomizedSearchCV 를 쓴다. 같은 시간에 더 넓은 공간을 살펴볼 수 있다.

과제

- cv= 값만 3, 5, 10 으로 바꿔 가며 평균 정확도를 출력한다. 코드는 거의 그대로이고 숫자 하나만 바뀐다. 어느 cv 값에서 정확도의 평균이 가장 높았는지, 가장 낮았는지, 가장 적절한 cv 값은 얼마인지 찾아보자.
- max_depth 후보에 4 와 5 를 추가해 다시 실행한다(나머지는 그대로). best_params_ 와 best_score_ 를 찾아보자.